

Implement Client Script & UI Policy (Incident)

Date: 03 October 2026

Team ID: 6ab75aba534c8646a9ce4107

INTRODUCTION:

1.1 Project Overview:

The Implement Client Script & UI Policy (Incident) project focuses on improving data entry and field behavior in ServiceNow Incident Management. Incident records require accurate and complete information for effective triage, routing, and resolution. This project demonstrates how UI Policies and Client Scripts can be used to dynamically control fields on the Incident form based on specific conditions.

The implementation includes features such as making fields mandatory, controlling field behavior, automatically setting values, validating information before saving, and preventing unauthorized changes through list editing. These controls help maintain consistent and accurate Incident records while improving the overall usability of the ServiceNow platform.

1.2 Purpose:

The purpose of this project is to enforce data integrity and consistency in Incident records using ServiceNow client-side controls. The project uses UI Policies and different types of Client Scripts, including onChange, onSubmit, and onCellEdit, to implement business rules directly at the user interface level.

The solution ensures that important fields are completed correctly, required values are automatically populated when necessary, invalid submissions are prevented, and restricted field changes are controlled. This helps improve the quality and reliability of Incident data and supports effective Incident Management.

2. IDEATION PHASE – PHASE 1

2.1 Problem Statement

Section	Content
I am (Customer)	A ServiceNow Incident Management user
I'm trying to	Create and update Incident records with complete and accurate information
But	Important fields may be left empty or entered incorrectly
Because	Users currently depend on manual checking and awareness of business rules
Which makes me feel	Frustrated and concerned about data accuracy, routing, and SLA compliance

Section	Content
I am (Customer)	A ServiceNow administrator or Incident manager
I'm trying to	Ensure Incident records follow defined business rules consistently
But	Users can submit incomplete records or make inappropriate field changes

Because	Field behavior and validation are not automatically enforced at the user-interface level
---------	--

Which makes me feel	Concerned about inconsistent Incident data and inefficient Incident management
---------------------	--

Final project statement:

Incident records require consistent and accurate data entry for effective triage, routing, and resolution. However, relying on manual checks can result in incomplete, inconsistent, or incorrect information. Therefore, UI Policies and Client Scripts are required to enforce conditional field behavior, mandatory fields, automatic updates, and save-time validation directly on the Incident form.

2.2 Empathy Map Canvas:

Your uploaded Empathy Map template is intended to capture the user's behaviours, attitudes, goals, and challenges from the user's perspective.

User: ServiceNow Incident User / Incident Management Team

SAYS

"I need to create the Incident quickly."

"Why can't I change the Urgency?"

THINKS

"I need to make sure all required information is entered."

"The system should tell me what information is required."

“I need to assign this Incident correctly.”	“I don't want to submit an Incident with missing information.”
---	--

“I want to update the Incident easily.”	“The system should prevent incorrect changes automatically.”
---	--

DOES

FEELS

Creates Incident records	Frustrated when validation errors occur unexpectedly
Selects Impact and Urgency	Concerned about entering incorrect information
Assigns Incidents to users/groups	Confident when the system guides the user
Updates Incident State	Frustrated when unauthorized list editing is possible
Saves and submits Incident records	Reassured when business rules are automatically enforced

PAINS

- Incomplete Incident information
- Incorrect field values
- Missing Assigned To information
- Manual validation effort
- Incorrect State changes through list editing
- Inconsistent Incident data

- Potential impact on reporting and SLA compliance

GAINS

- Automatic field validation
- Clear mandatory-field enforcement
- Automatic Urgency updates
- Controlled field behavior
- Prevention of incorrect list edits
- Consistent Incident records
- Better data quality and usability

USER GOALS

1. Create complete Incident records.
2. Enter accurate Incident information.
3. Understand which fields are mandatory.
4. Update Incidents correctly.
5. Save Incidents without validation problems.
6. Follow Incident-management business rules automatically.

This matches the purpose of the project: using UI Policies and Client Scripts to dynamically control fields and prevent invalid Incident submission.

2.3 Brainstorming

Your Brainstorming template defines three stages: Team Gathering & Problem Selection → Brainstorm/Idea Listing & Grouping → Idea Prioritization.

Step 1 – Team Gathering, Collaboration and Select the Problem Statement

Selected Problem

How can we ensure that Incident records contain complete and accurate information while automatically enforcing business rules at the user-interface level?

Step 2 – Brainstorm, Idea Listing and Grouping

💡 Brainstormed Ideas

Category	Idea
Validation	Prevent saving an Incident when Assigned To is missing
Validation	Display an error message for invalid submissions
Automation	Automatically set Urgency when Impact is High
Field Control	Make Assignment Group mandatory for High Impact
Field Control	Make Urgency read-only for High Impact
Field Control	Reverse field behavior when Impact changes
List Control	Prevent State changes through direct list editing
User Guidance	Display informative messages to users
Data Quality	Enforce consistent Incident information
Testing	Test High Impact, successful save, reverse condition, and list editing

Priority	Idea	Purpose
1	UI Policy – High Impact Control	Apply conditional Incident field rules
2	Make Assignment Group mandatory	Ensure High Impact Incidents have proper assignment
3	Urgency read-only for High Impact	Prevent inappropriate manual changes
4	onChange Client Script	Automatically set Urgency to High
5	onSubmit Client Script	Prevent saving without Assigned To
6	onCellEdit Client Script	Prevent State changes through list editing
7	Reverse if false	Restore normal field behavior when Impact changes

These selected ideas directly correspond to the implementation activities in your project, including the High Impact Control UI Policy, Urgency control, onChange automation, onSubmit validation, and onCellEdit restriction.

3. REQUIREMENT ANALYSIS PHASE - 2

3.1 Customer journey map

SERVICENOW · INCIDENT MANAGEMENT

Implement Client Script & UI Policy (Incident)

Customer journey map - how High-impact Incident controls are discovered, built, tested and extended

TEAM ID
6ab75aba534c8646a9ce4107

Based on project Tasks 3-6: UI Policy, UI Policy Action, onChange / onSubmit / onCellEdit Client Scripts, and testing activities.

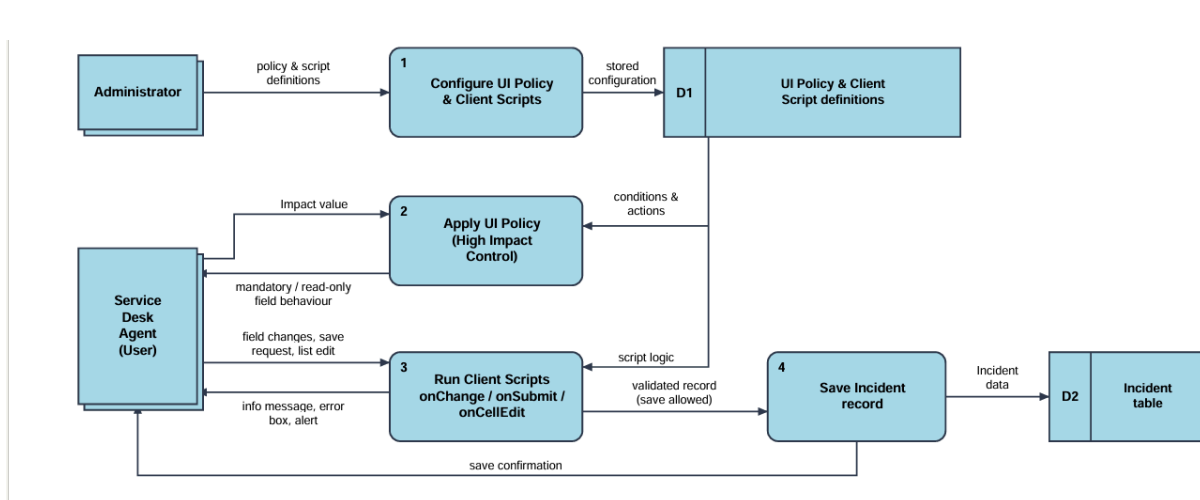
SCENARIO	Entice	Enter	Engage	Exit	Extend															
Configuring, validating and testing client-side controls on High-impact Incidents	How does someone initially become aware of this need?	What do people experience as they begin the configuration?	In the core moments of the build, what happens?	What do people experience as the testing finishes?	What happens after the project is delivered?															
Steps	Spot data issues: Incidents arrive with missing or inconsistent fields	Link to SLAs: Add data hints, routing, reporting and SLA compliance	Define rules: High-impact incidents need an owner and a controlled urgency	Pick the controls: UI Policy for Submitter, Client Scripts for logic and validation	Log in: Open the instance with admin or configuration access	Create UI Policy: System UI - UI Policies - New High Impact Control on Incident, Active	Set condition: Impact > 3 - High Assignment mandatory, Reverse if false = true	Add Urgency action: Urgency: Read-only = true, Visible left unchanged	onChange script: Impact changes to 3, Urgency to 1 and leaves an info message	onSubmit script: Block save if Impact is High and Assigned To is empty	onCellEdit script: Alert and cancel any State edit made from a list	Test mandatory rule: New Incident, Impact = High, Assigned To empty, Submit	Test successful save: Fill Assigned To and submit, Incident saves with no errors	Test reverse condition: Change Impact High to Medium, fields update and record saves	Test list-edit block: Double-click on the incident list, alert appears	Test form update: Change State on the form and click Update, change is saved	Cleaner data: Incidents are created with complete, valid information	Reliable reporting: Consistent fields support reports and SLA tracking	Reuse the pattern: Same UI Policy + script approach works on other forms	Wrap up: Document results in the project conclusion
Interactions	Places: Incident list, reports, service desk agents, process owner	Things: SLA and reporting view, People: service desk manager	People: process owner and admin, agreeing the business rules	Things: project team, SLAs, UI Policies, Client Scripts, form validation	Places: ServiceNow view, People: incident manager, urgency, config role	Places: System UI - UI Policies - New	Things: UI Policy form, Conditions section and Reverse if false	Things: UI Policy form, Urgency: Read-only = true	Places: System UI - Client Scripts - New, Things: onChange, field impact	Things: Client Script type, onsubmit, g, for m, onsubmit, g, for m, onsubmit, g, for m	Things: Client Script type, onCellEdit, field impact, callback, false	Places: Incident form, Create new form	Places: Incident form, Open existing High-impact record	Places: Incident form, Open existing High-impact record	Places: Incident form, Open existing High-impact record	Places: Incident form, Open existing High-impact record	Places: Incident form, Open existing High-impact record	Places: Incident form, Open existing High-impact record	Things: project documentation	
Goals & motivations	Help me see where manual data entry goes wrong	Help me understand the impact of SLAs	Help me define clear rules for high-impact incidents	Help me choose the right control for each rule	Help me get into the right configuration area	Help me set up a policy that applies only to incident	Help me trigger controls only after impact = High	Help me stop users from changing urgency by mistake	Help me avoid users setting urgency by hand	Help me avoid incidents saved without an owner	Help me avoid risky State changes in list edit	Help me confirm the rule are nearly enforced	Help me confirm valid records still save clearly	Help me avoid fields staying blocked after impact drops	Help me confirm list with are blocked	Help me confirm normal form, options still work	Help me trust that incident reports are complete	Help me report accurately and meet SLA	Help me avoid rebuilding the form from scratch	Help me keep the solution light and easy to maintain
Positive moments	Rules are short and easy to state																			
Negative moments	Relating on user awareness leads to incomplete or incorrect data	Reporting accuracy and SLA compliance suffer																		
Areas of opportunity	How might we make each user better? What else do we know? What have others suggested?																			

4. Project Design Phase-II Data Flow Diagram & User Stories

3.2 Data Flow Diagrams:

A Data Flow Diagram (DFD) is a visual representation of the information flows within a system. It shows how data enters and leaves the system, what changes the information, and where data is stored. The Level 0 DFD below shows how the UI Policy and Client Scripts control data entry on ServiceNow Incident records. DFD Level 0: Incident client-side controls

DFD Level 0: Incident client-side controls:



Flow

1. Administrator logs in and creates the High Impact Control UI Policy, the Urgency action and the three Client Scripts; the definitions are stored (D1).
2. Agent opens an Incident form. When Impact is 1 - High, the UI Policy makes Assignment group mandatory and Urgency read-only, and reverses both when Impact changes.
3. The onChange script sets Urgency to High and shows an info message when Impact changes to High.
4. On Submit, the onSubmit script blocks the save if Impact is High and Assigned To is empty, and shows an error on the field.
5. The onCellEdit script shows an alert and cancels any State edit made from the list view.
6. Valid records are saved to the Incident table (D2) and a confirmation is returned to the agent

User Stories

User stories for the Incident client-side controls.

User Type	Functional Requirement (Epic)	User Story Number	User Story / Task	Acceptance criteria	Priority	Release
Administrator	UI Policy	USN-1	As an administrator, I can create the High Impact Control UI Policy on the Incident table with the condition Impact is 1 - High and Reverse if false enabled.	The policy is active and applies only when Impact is High.	High	Sprint-1
		USN-2	As an administrator, I can make Assignment group mandatory through the UI Policy when Impact is High.	Assignment group is mandatory for High impact and returns to normal when Impact changes.	High	Sprint-1
		USN-3	As an administrator, I can add a UI Policy Action that makes Urgency read-only when Impact is High.	Urgency stays visible but read-only for High impact; it is editable again otherwise.	High	Sprint-1
	Client Scripts	USN-4	As an administrator, I can add an onChange script that sets Urgency to High when Impact changes to High.	Urgency is set to 1 and an info message is shown.	High	Sprint-2
		USN-5	As an administrator, I can add an onSubmit script that blocks saving when Impact is High and Assigned To is empty.	The record is not saved and an error appears on Assigned To.	High	Sprint-2
		USN-6	As an administrator, I can add an onCellEdit script that blocks State changes from the list view.	An alert is shown and the State value is unchanged.	Medium	Sprint-2
Service Desk Agent	Incident form	USN-7	As an agent, when I set Impact to High, Urgency updates to High automatically and cannot be edited.	Urgency shows High and is read-only after Impact is set to High.	High	Sprint-2
		USN-8	As an agent, I am stopped from saving a High-impact Incident without Assigned To and told why.	Save is blocked and an error message is shown on Assigned To.	High	Sprint-3
		USN-9	As an agent, I can save a High-impact Incident once Assigned To is filled.	The Incident saves without errors.	High	Sprint-3
		USN-10	As an agent, when I lower Impact to Medium, the extra controls are removed and I can save normally.	Assigned To is no longer mandatory, Urgency is editable and the record saves.	Medium	Sprint-3
	Incident list	USN-11	As an agent, I am told State cannot be changed from the list, and I can still change State on the form.	List edit shows an alert and keeps the value; a form update of State saves.	Medium	Sprint-3
Service Delivery Manager	Reporting	USN-12	As a manager, I can rely on Incidents having complete, valid data so reports and SLA tracking are accurate.	High-impact Incidents always carry an owner and a consistent Urgency.	Medium	Sprint-3

3.3.Project Design Phase-II Solution Requirements (Functional & Non-functional)

Following are the functional requirements of the proposed solution

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	UI Policy Configuration (High Impact Control)	Create the High Impact Control UI Policy on the Incident table (Active)
		Trigger the policy only when Impact is 1 - High
		Make Assignment group mandatory when the condition is met
		Enable Reverse if false so changes are undone when the condition is not met
FR-2	UI Policy Action - Urgency	Make Urgency read-only when Impact is High
		Keep Urgency visible on the form (Visible left unchanged)
		Restore Urgency editing when Impact changes to another value
FR-3	onChange Client Script (Auto-set Urgency)	Run when the Impact field changes, ignoring form load and empty values
		Set Urgency to 1 (High) when Impact changes to High
		Show an info message explaining the automatic update
FR-4	onSubmit Client Script (Save Validation)	Block saving when Impact is High and Assigned To is empty
		Show an error message on the Assigned To field
		Allow the save once Assigned To is filled
FR-5	onCellEdit Client Script (List Edit Control)	Block State changes made directly from the Incident list
		Show an alert asking the user to open the Incident
		Allow State changes made through the Incident form
FR-6	Testing & Validation	Test mandatory enforcement (High impact, Assigned To empty)
		Test successful save, reverse condition (High to Medium) and form-based update
		Test list edit blocking on the Incident list

Non-functional Requirements: Following are the non-functional requirements of the proposed solution

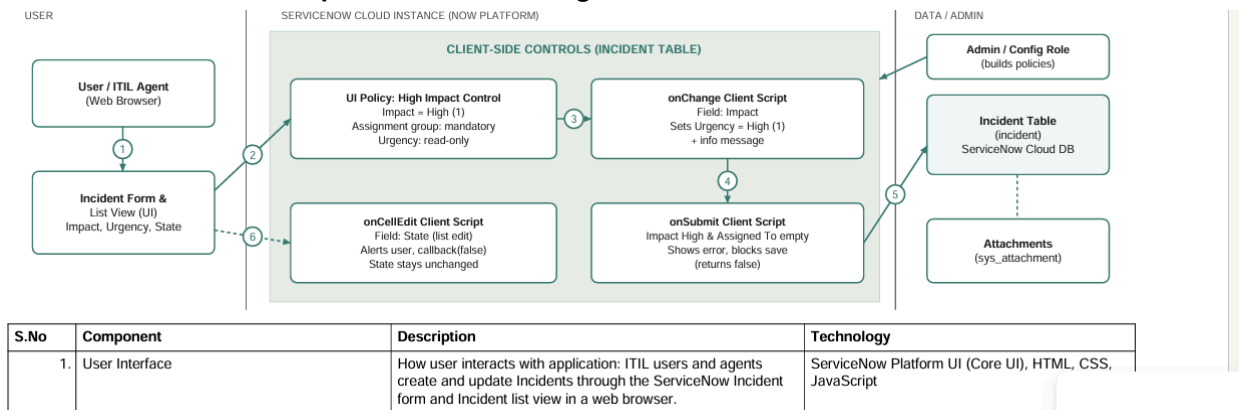
FR No.	Non-Functional Requirement	Description
NFR-1	Usability	Fields respond immediately to Impact, and clear info, error and alert messages tell users what happened and what to do next.
NFR-2	Security	Only users with administrative or configuration access can create UI Policies and Client Scripts. State changes from the list view are restricted. These client-side controls apply to the user interface only.
NFR-3	Reliability	Reverse if false returns fields to normal when the condition ends, and save-time validation applies the same rule on every submit.
NFR-4	Performance	Scripts are lightweight, run only on the Incident table and the relevant field, and skip form load, so the form stays fast.
NFR-5	Availability	Controls run inside the ServiceNow instance with no external dependency and are enforced as soon as the policy or script is set to Active.
NFR-6	Scalability	The UI Policy and Client Script pattern can be reused for other conditions, fields and tables, and conditions can be changed without rebuilding the solution.

Technology Stack (Architecture & Stack)

Technical Architecture:

The Deliverable shall include the architectural diagram as below and the information as per the table1 & table 2

Example: Implement Client Script & UI Policy (Incident) – enforcing Incident data integrity in ServiceNow Table-1 : Components & Technologies



2.	Application Logic-1	UI Policy "High Impact Control" (table: Incident; condition: Impact is 1 – High). Makes Assignment group mandatory and Urgency read-only; Reverse if false = true restores fields when Impact changes.	ServiceNow UI Policy & UI Policy Actions
3.	Application Logic-2	onChange Client Script "Auto set urgency for high impact" on field Impact. When Impact = 1, sets Urgency = 1 and displays an info message.	ServiceNow Client Script (onChange), JavaScript, g_form API
4.	Application Logic-3	onSubmit Client Script blocks save when Impact is High and Assigned To is empty. onCellEdit Client Script blocks State changes made from the list view.	ServiceNow Client Script (onSubmit, onCellEdit), JavaScript
5.	Database	Incident records (table: incident) store Impact, Urgency, Assignment group, Assigned To and State values validated by the client-side rules.	ServiceNow relational database (Incident table)
6.	Cloud Database	Managed database service hosted with the ServiceNow cloud instance; no separate database setup needed.	ServiceNow Cloud (Now Platform) managed database
7.	File Storage	Optional attachments on Incident records are stored in the platform attachment tables.	ServiceNow Attachment storage (sys_attachment)
8.	External API-1	No third-party API is required. Optional integration with other systems can use the platform's built-in REST interface.	ServiceNow Table REST API (optional)
9.	External API-2	Not used in this project.	Not applicable
10.	Machine Learning Model	Not used. Data integrity is enforced through rule-based UI Policies and Client Scripts.	Not applicable
11.	Infrastructure (Server / Cloud)	Application Deployment on Cloud (ServiceNow instance) Local Server Configuration: None; only a modern web browser is needed. Cloud Server Configuration: ServiceNow-hosted instance; admin/configuration role to create policies and scripts.	ServiceNow Cloud (Now Platform) instance

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1.	Open-Source Frameworks	Open web standards used for client-side logic and UI; the ServiceNow platform itself is proprietary.	JavaScript, HTML, CSS
2.	Security Implementations	Role-based access control and ACLs, admin-only configuration, HTTPS/TLS access, mandatory-field and save-time validation to prevent incomplete data.	ServiceNow ACLs, RBAC, TLS (HTTPS)
3.	Scalable Architecture	Multi-tier platform (browser, application nodes, database). Validation runs in the user's browser, and one policy applies to every Incident form.	ServiceNow multi-tier Now Platform
S.No	Characteristics	Description	Technology
4.	Availability	Cloud-hosted instance with redundant infrastructure managed by ServiceNow; the logic is stored on the platform, so it is available whenever the instance is.	ServiceNow cloud data centers (managed failover)
5.	Performance	Lightweight scripts run only on specific events (Impact change, save, list edit), so there are no extra server round trips for validation.	Client-side execution, browser caching

References

<https://c4model.com>

<https://developer.ibm.com/patterns/online-order-processing-system-during-pandemic/>

<https://www.ibm.com/cloud/architecture> <https://aws.amazon.com/architecture>

<https://medium.com/the-internal-startup/how-to-draw-useful-technical-architecture-diagrams-2d20c9fda90d>

Project design phase- 3

Problem-Solution fit canvas 2.0

Enforce complete, consistent and valid data on ServiceNow Incident records at the UI level using UI Policies and Client Scripts, so incidents are triaged, routed and resolved correctly.

Define CS, fit into CC	1. CUSTOMER SEGMENT(S) Who is your customer? I.e. working parents of 0-5 y.o. kids - Service Desk / ITIL agents who create and update Incident records - Incident managers and fulfiller groups who triage and route incidents - ServiceNow admins and developers who configure form behaviour - Reporting and SLA owners who depend on accurate incident data	6. CUSTOMER CONSTRAINTS What constraints prevent your customers from taking action or limit their choices of solutions? I.e. spending power, budget, no cash, network connection, available devices. - Relying only on user awareness and manual checks is unreliable - Rules must run on the form and list UI without slowing agents down - Configuration needs admin / configuration access to the instance - Solution limited to client-side controls (UI Policy, Client Scripts) - Must stay lightweight and fit a short-duration micro project	5. AVAILABLE SOLUTIONS Which solutions are available to the customers when they face the problem or need to get the job done? What have they tried in the past? What pros & cons do these solutions have? I.e. pen and paper is an alternative to digital notetaking - Manual review, reminders and training: low cost but inconsistent and error-prone - Always-mandatory dictionary fields: not conditional, so too rigid - Server-side Business Rules / Data Policies: check only after submit, slower feedback - Cleaning data later via reports: reactive; hurts SLA and reporting accuracy	Explore AS, differentiate
	2. JOBS-TO-BE-DONE / PROBLEMS Which jobs-to-be-done (or problems) do you address for your customers? There could be more than one; explore different sides. - Make Assignment group mandatory when Impact is High - Auto-set Urgency to High (and lock it) when Impact is High - Block saving a High-impact incident when Assigned To is empty, with a clear error message - Prevent State from being changed through list (inline) editing - Reverse field behaviour automatically when Impact is no longer High	9. PROBLEM ROOT CAUSE What is the real reason that this problem exists? What is the back story behind the need to do this job? I.e. customers have to do it because of the change in regulations. - Data entry depends on user awareness and manual checks - No UI-level enforcement of conditional business rules - Result: incomplete, inconsistent or incorrect incident data - Impact: poor triage and routing, inaccurate reporting, missed SLAs, lower service quality - List editing lets users bypass the rules applied on the form	7. BEHAVIOUR What does your customer do to address the problem and get the job done? I.e. directly related: find the right solar panel installer, calculate usage and benefits; indirectly associated: customers spend free time on volunteering work (I.e. Greenpeace) - Agent opens a new Incident, sets Impact and fills in fields quickly - Assigned To / Assignment group are sometimes left blank and Urgency set inconsistently - State is edited inline from the list for speed - Admin builds the UI Policy "High Impact Control" plus onChange, onSubmit and onCellEdit Client Scripts - Admin tests mandatory enforcement, successful save, reverse condition, list blocking and form update	
3. TRIGGERS What triggers customers to act? I.e. seeing their neighbour installing solar panels, reading about a more efficient solution in the news. - Impact set to High on an Incident - Clicking Submit / Update on the form - Double-clicking State in the list view - Poor reports or SLA misses exposing bad data	10. YOUR SOLUTION If you are working on an existing business, write down your current solution first, fill in the canvas, and check how much it fits reality. If you are working on a new business proposition, then keep it blank until you fill in the canvas and come up with a solution that fits within customer limitations, solves a problem and matches customer behaviour. - Implemented on the ServiceNow Incident table: - UI Policy "High Impact Control" (Impact = 1 - High): Assignment group mandatory: Urgency read-only; Reverse if false = true - onChange Client Script: Impact = High sets Urgency = High and shows an info message - onSubmit Client Script "Prevent save if Assigned To missing": shows error and blocks save - onCellEdit Client Script: alerts user and blocks State change from the list - Tested: mandatory enforcement, successful save, reverse condition, list-edit blocking, form-based update	8. CHANNELS of BEHAVIOUR 8.1 ONLINE What kind of actions do customers take online? Extract online channels from #7 - Create / update incidents on the ServiceNow Incident form - See instant info, error and alert messages on screen - Admin configures via System UI > UI Policies / Client Scripts 8.2 OFFLINE What kind of actions do customers take offline? Extract offline channels from #7 and use them for customer development. - Service desk stand-ups and team briefings on data-entry standards - Training and guidance on required fields for High-impact incidents	Extract online & offline CH of BE	
4. EMOTIONS: BEFORE / AFTER How do customers feel when they face a problem or a job and afterwards? I.e. lost, insecure = confident, in control - use it in your communication strategy & design. - Before: frustrated, uncertain, doing rework; no control over data quality - After: confident and in control; clear on-screen messages guide correct entry	EM			

Problem-Solution fit canvas is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 license
Created by Daria Nepriakhina / Amaltama.com

Proposed solution:

Proposed Solution Template:

Project team shall fill the following information in the proposed solution template

S.No	Parameter	Description
1	Problem Statement (Problem to be solved)	Incident records need consistent and accurate data for effective triage, routing, and resolution. Manual checks can lead to incomplete or incorrect submissions, affecting reporting accuracy, SLA compliance, and service quality.

2	Idea / Solution Description	ServiceNow client-side controls are implemented on the Incident form. A UI Policy – “High Impact Control” makes Assignment Group mandatory and Urgency read-only when Impact = 1 (High). Client Scripts include onChange to set Urgency to High, onSubmit to prevent saving when Assigned To is empty, and onCellEdit to prevent State edits from list view.
3	Novelty / Uniqueness	Combines a UI Policy with three types of Client Scripts—onChange, onSubmit, and onCellEdit—to control form submission and list editing. The solution is lightweight, requires no server-side code, automatically reverts when conditions change, and prevents list-edit loopholes.
4	Social Impact / Customer Satisfaction	Complete and accurate incident data helps speed up triage and resolution. Clear on-screen messages guide agents, reduce errors and rework, improve SLA compliance, and provide more reliable support to end users.
5	Business Model (Revenue Model)	Can be delivered as a reusable ServiceNow configuration or consulting package. Revenue can come from one-time implementation fees, client-specific customization, and ongoing support and maintenance subscriptions. Organizations benefit from reduced rework and fewer SLA breaches.
6	Scalability of the Solution	The same approach can be extended to Change, Problem, and Request tables, as well as other fields and conditions. It can be packaged as an Update Set or Scoped Application for deployment across multiple instances. Client-side logic also minimizes additional server load.

Solution Architecture:

Solution architecture is a complex process – with many sub-processes – that bridges the gap between business problems and technology solutions. Its goals are to:

- Find the best tech solution to solve existing business problems.
- Describe the structure, characteristics, behavior, and other aspects of the software to project stakeholders.
- Define features, development phases, and solution requirements.
- Provide specifications according to which the solution is defined, managed, and delivered.

Solution Architecture Diagram:

Example - Solution Architecture Diagram:

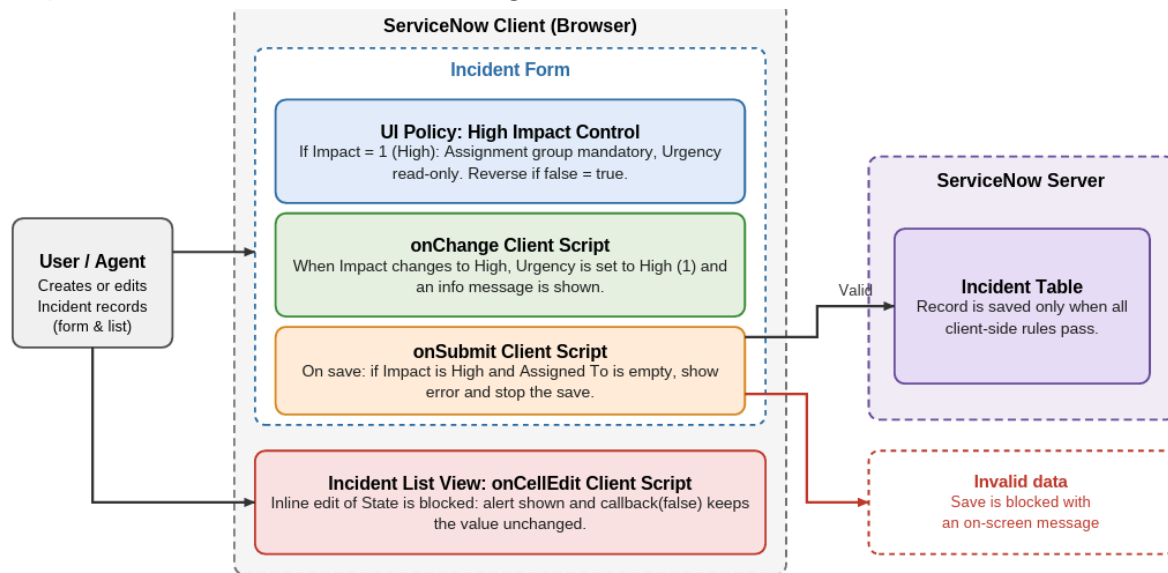


Figure 1: Architecture and data flow of the Incident Client Script & UI Policy solution

Reference: <https://www.servicenow.com/docs>.

5. Project Planning Phase

Product Backlog, Sprint Schedule, and Estimation

Sprint	Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority	Team Members
Sprint -1	UI Policy Configuration	USN-1	As an administrator, I can create a High Impact Control UI Policy on the Incident table so that additional controls are applied when Impact is High.	2	High	Team
Sprint -1	UI Policy Configuration	USN-2	As an administrator, I can make the Assignment Group field mandatory when the Incident Impact is High.	2	High	Team

Sprint -1	UI Policy Configuration	USN-3	As an administrator, I can configure Reverse if false so that the UI Policy changes are reverted when the condition is no longer satisfied.	1	Medium	Team
Sprint -2	UI Policy Action	USN-4	As an administrator, I can configure the Urgency field as read-only when Impact is High.	2	High	Team
Sprint -2	Client Script – onChange	USN-5	As an administrator, I can create an onChange Client Script on Impact that automatically sets Urgency to High when Impact is High.	3	High	Team
Sprint -2	Client Script – onSubmit	USN-6	As an administrator, I can prevent an Incident from being saved when Impact is High and Assigned To is empty.	3	High	Team
Sprint -3	Client Script – onCellEdit	USN-7	As an administrator, I can prevent users from changing the State directly through list editing.	3	Medium	Team
Sprint -3	Testing & Validation	USN-8	As a tester, I can verify that mandatory-field validation, urgency auto-update, read-only behavior, and State restrictions work correctly.	3	High	Team
Sprint -3	Testing & Validation	USN-9	As a tester, I can verify that Incident records can be successfully saved when all required conditions are satisfied.	2	High	Team

Sprint -3	Testing & Validation	USN-10	As a tester, I can verify that changing Impact from High to Medium reverses the UI Policy behavior.	2	Mediu m	Team
--------------	-------------------------	--------	---	---	------------	------

Total Story Points: 23

Sprint Schedule

Sprint	Total Story Points	Duration	Sprint Start Date	Sprint End Date	Planned Story Points Completed	Sprint Release Date
Sprint-1	5	2 Days	03 Oct 2026	04 Oct 2026	5	04 Oct 2026
Sprint-2	8	2 Days	05 Oct 2026	06 Oct 2026	8	06 Oct 2026
Sprint-3	10	3 Days	07 Oct 2026	09 Oct 2026	10	09 Oct 2026

The template's tracker format includes total story points, duration, start/end dates, completed points, and actual release date.

Sprint-1 — UI Policy Configuration

Goal: Configure the High Impact UI Policy.

Tasks:

1. Create High Impact Control UI Policy.
2. Set Table = Incident.
3. Set condition:
 - o Field = Impact
 - o Operator = is
 - o Value = 1 – High
4. Configure Assignment Group as mandatory.
5. Enable Reverse if false.
6. Test the policy with High and non-High Impact values.

The source project specifically defines the High Impact Control policy and its Incident/Impact configuration.

Sprint-2 — Client Scripts & UI Policy Action

Goal: Implement dynamic Incident form behavior.

Tasks:

1. Configure Urgency UI Policy Action.
2. Make Urgency read-only when Impact = High.
3. Create onChange Client Script for Impact.
4. Automatically set Urgency to High.
5. Create onSubmit Client Script.
6. Prevent saving when:
 - Impact = High
 - Assigned To is empty.

The uploaded project specifies the onChange script for automatically setting Urgency and the onSubmit validation for Assigned To.

Sprint-3 — List Editing & Testing

Goal: Complete the onCellEdit control and validate the complete configuration.

Tasks:

1. Create onCellEdit Client Script.
2. Apply it to the Incident State field.

3. Prevent direct State modification through list editing.

The screenshot shows the ServiceNow Incident form for incident INC0010003. The form includes fields for Number, Caller, Category, Subcategory, Service, Service offering, Configuration item, Channel, State, Impact, Urgency, Priority, Assignment group, and Assigned to. A red error message at the top states: "The following mandatory fields are not filled in: Short description". Another red error message near the Assigned to field states: "Assigned To is mandatory for High impact incidents." The form also includes a "Short description" field with the value "Hi" and a "Description" field.

4. Test mandatory-field enforcement.
5. Test successful Incident save.
6. Test Reverse if false.
7. Test State list-edit blocking.
8. Test State update through the Incident form.

The source project defines the onCellEdit requirement and five testing activities covering these behaviors.

Project Tracker

Sprint	Total Story Points	Duration	Planned Start	Planned End	Completed Story Points	Release Date
Sprint-1	5	2 Days	03 Oct 2026	04 Oct 2026	5	04 Oct 2026
Sprint-2	8	2 Days	05 Oct 2026	06 Oct 2026	8	06 Oct 2026
Sprint-3	10	3 Days	07 Oct 2026	09 Oct 2026	10	09 Oct 2026
Total	23	7 Days	03 Oct 2026	09 Oct 2026	23	09 Oct 2026

Velocity

Based on the planned schedule:

Total Story Points = 23

Total Duration = 7 days

Average Velocity = $23 \div 7 = 3.29$ story points/day

The template defines velocity as the team's completed story points relative to the sprint/iteration duration.

Burndown Chart Data

Day	Planned Remaining Story Points
Day 1	20
Day 2	18
Day 3	15
Day 4	12
Day 5	9
Day 6	5
Day 7	0

A burndown chart represents work remaining versus time, which is the format specified in the provided template.

Final Project Structure

Implement Client Script & UI Policy (Incident)

→ Sprint 1: UI Policy

→ Sprint 2: UI Policy Action + onChange + onSubmit

→ Sprint 3: onCellEdit + Testing

→ 23 Story Points

→ 7 Days Planned Duration

This keeps your ServiceNow project content intact while converting it into the Product Backlog + Sprint Planning format of the uploaded template.

6.FUNCTIONAL AND PERFORMANCE TESTING

Test Case	Testing Activity	Expected Result	Status
TC-01	Set Impact = High and leave Assigned To empty, then click Submit.	Incident should not be saved and an error message should be displayed.	Pass
TC-02	Set Impact = High and select a value in Assigned To, then click Submit.	Incident should save successfully without errors.	Pass
TC-03	Set Impact = High.	Urgency should automatically be set to High and become read-only.	Pass
TC-04	Change Impact from High to Medium.	Assigned To should no longer be mandatory and Urgency should become editable.	Pass
TC-05	Open Incident → All and try to edit State directly from the list.	Alert should appear and the State value should remain unchanged.	Pass
TC-06	Open the Incident form and change State from the form.	State change should be saved successfully.	Pass

Testing Summary

The testing verifies the complete Client Script and UI Policy configuration:

- UI Policy: High Impact condition works correctly.
- Assignment Group: Mandatory behavior is enforced.
- Urgency: Automatically updated and made read-only for High Impact.
- onSubmit Client Script: Prevents saving when Assigned To is missing.
- onCellEdit Client Script: Blocks State changes through list editing.
- Reverse if false: Restores normal field behavior when Impact changes from High to Medium.
- Form-based State update: Continues to work normally.

Testing Result: All test cases passed.

7. RESULTS

7.1 Output Screenshots

dev424338.service-now.com/now/nav/ui/classic/params/target/incident.do%3Fsys_id%3D-1%26sysparm_clear_stack%3Dtrue

servicenow All Favorites History Workspaces Incident - Create INC0010003 Search Submit Resolve

Incident New record

Urgency set to High for High impact incident.

The following mandatory fields are not filled in: Short description

Number INC0010003

* Caller Abel Tuter

Category Inquiry

Subcategory -- None --

Service

Service offering

Configuration item

Channel -- None --

State New

Impact 1 - High

Urgency 1 - High

Priority 1 - Critical

* Assignment group CAB Approval

Assigned to

Assigned To is mandatory for High impact incidents.

* Short description Hii

Description

Related Search Results

dev424338.service-now.com/now/nav/ui/classic/params/target/incident.do%3Fsys_id%3D2b18fd6b2f670310e4e0b7aa6fa4e367%26sysparm_view%3D%26sysparm_domain%3Dnull%26sysparm...

servicenow All Favorites History Workspaces Admin Incident - INC0010002 Search Discuss Follow Update Resolve Delete Responses

Incident INC0010002

Urgency set to High for High impact incident.

Number INC0010002

* Caller Abel Tuter

Category Inquiry

Subcategory -- None --

Service

Service offering

Configuration item

Channel -- None --

State New

Impact 1 - High

Urgency 1 - High

Priority 1 - Critical

* Assignment group CAB Approval

Assigned to

* Short description Practise

Description

Related Search Results

Notes Related Records Resolution Information

Watch list

Work notes list

dev424338.service-now.com/now/nav/ui/classic/params/target/incident.do%3Fsys_id%3D2b18fd6b2f670310e4e0b7aa6fa4e367%26sysparm_view%3D%26sysparm_domain%3Dnull%26syspar...

servicenowAllFavoritesHistoryWorkspacesAdminIncident - INC0010002Search

IncidentINC0010002DiscussFollowUpdateResolveDeleteResponses

Urgency set to High for High impact incident.

NumberINC0010002

* CallerAbel Tuter

CategoryInquiry

Subcategory-- None --

Service

Service offering

Configuration item

* Short descriptionPractise

Description

Channel-- None --

StateNew

Impact1 - High

Urgency1 - High

Priority1 - Critical

* Assignment groupCAB Approval

Assigned to

Related Search Results >

* NotesRelated RecordsResolution Information

Watch list

Work notes list

Number	Opened	Short description	Caller	Priority	State	Category	Assignment group	Assigned to	Updated	Updated by
INC0010002	2026-10-01 11:35:35	Practise	Abel Tuter	4 - Low	New		(empty)	(empty)	2026-10-01 11:37:36	admin
INC0009009	2018-08-30 01:06:16	Unable to access the shared folder.	David Miller	4 - Low	New	Inquiry	(empty)	(empty)	2018-12-12 23:30:24	admin
INC0009005	2018-08-31 21:35:21	Email server is down.	David Miller	1 - Critical	New	Collaboration Tools	(empty)	(empty)	2018-12-12 23:18:55	admin
INC0009004	2018-09-01 06:13:30	Defect tracking tool is down.	David Miller	3 - Moderate	Closed	Software	(empty)	(empty)	2026-09-22 20:48:23	system
INC0009003	2018-08-30 02:17:32	Cannot sign into the company portal app	David Miller	3 - Moderate	Closed	Inquiry	(empty)	(empty)	2018-12-12 23:39:53	admin
INC0009002	2018-09-16 05:49:23	My computer is not detecting the headphone device	David Miller	3 - Moderate	Closed	Hardware	(empty)	(empty)	2026-09-22 20:48:21	system
INC0009001	2018-09-11 20:56:26	Unable to post content on a Wiki page	David Miller	3 - Moderate	New	Inquiry	(empty)	(empty)	2018-12-12 23:32:42	admin
INC0008112	2019-07-29 11:48:43	Assessment : ATF Assessor	survey user	4 - Low	New	Inquiry	(empty)	(empty)	2019-07-29 11:49:28	admin
INC0008111	2019-07-22 14:04:57	ATF : Test1	System Administrator	5 - Planning	New	Inquiry	(empty)	(empty)	2019-07-22 14:05:48	admin
INC0008001	2021-01-15 13:04:14	ATF:TEST2	survey user	4 - Low	New	Inquiry	(empty)	(empty)	2021-01-21 15:31:42	admin

Number	Opened	Short description	Caller	Priority	State	Category	Assignment group	Assigned to	Updated	Updated by
INC0010002	2026-10-01 11:35:35	Practise	Abel Tuter	4 - Low	Resolved		(empty)	(empty)	2026-10-01 11:37:36	admin
INC0009009	2018-08-30 01:06:16	Unable to access the shared folder.	David Miller	4 - Low	New	Inquiry	(empty)	(empty)	2018-12-12 23:30:24	admin
INC0009005	2018-08-31 21:35:21	Email server is down.	David Miller	1 - Critical	New	Collaboration Tools	(empty)	(empty)	2018-12-12 23:18:55	admin
INC0009004	2018-09-01 06:13:30	Defect tracking tool is down.	David Miller	3 - Moderate	Closed	Software	(empty)	(empty)	2026-09-22 20:48:23	system
INC0009003	2018-08-30 02:17:32	Cannot sign into the company portal app	David Miller	3 - Moderate	Closed	Inquiry	(empty)	(empty)	2018-12-12 23:39:53	admin
INC0009002	2018-09-16 05:49:23	My computer is not detecting the headphone device	David Miller	3 - Moderate	Closed	Hardware	(empty)	(empty)	2026-09-22 20:48:21	system
INC0009001	2018-09-11 20:56:26	Unable to post content on a Wiki page	David Miller	3 - Moderate	New	Inquiry	(empty)	(empty)	2018-12-12 23:32:42	admin
INC0008112	2019-07-29 11:48:43	Assessment : ATF Assessor	survey user	4 - Low	New	Inquiry	(empty)	(empty)	2019-07-29 11:49:28	admin
INC0008111	2019-07-22 14:04:57	ATF : Test1	System Administrator	5 - Planning	New	Inquiry	(empty)	(empty)	2019-07-22 14:05:48	admin
INC0008001	2021-01-15 13:04:14	ATF:TEST2	survey user	4 - Low	New	Inquiry	(empty)	(empty)	2021-01-21 15:31:42	admin

8. ADVANTAGES & DISADVANTAGES:

Project Advantages

1. **Improves Data Accuracy** – Ensures Incident records contain the required information before submission.

2. **Reduces User Errors** – Prevents users from saving incomplete Incident records.
3. **Automatic Field Updates** – Urgency is automatically set to High when Impact is High.
4. **Dynamic Form Behavior** – Fields can become mandatory or read-only based on conditions.
5. **Better Data Consistency** – Standardizes how Incident records are entered and maintained.
6. **Improves User Experience** – Provides immediate feedback and validation on the form.
7. **Lightweight Implementation** – Uses Client Scripts and UI Policies without requiring complex customization.
8. **Easy to Test and Maintain** – Each rule can be tested independently.

Project Disadvantages

1. **Client-Side Dependency** – Client Scripts mainly control behavior on the user interface.
2. **Browser Dependency** – Incorrect or unexpected client-side behavior can affect the user experience.
3. **Maintenance Required** – Changes to Incident fields or business requirements may require updating the scripts.
4. **Limited Server-Side Enforcement** – Client-side validation alone should not be considered sufficient for all data-security requirements.
5. **Multiple Rules Can Conflict** – Several UI Policies and Client Scripts acting on the same field can create unexpected behavior.
6. **Testing Required** – Different Impact values, form actions, and list-edit scenarios must be tested carefully.

Conclusion:

The **Implement Client Script & UI Policy (Incident)** project demonstrates how ServiceNow UI Policies and Client Scripts can be used together to improve data integrity and control user interactions on Incident forms. The implementation dynamically makes fields mandatory, automatically updates values, controls field behavior, and prevents incorrect record submission based on defined conditions.

The use of **onChange, onSubmit, and onCellEdit Client Scripts**, along with UI Policy Actions, ensures that Incident records follow the required business rules. The project also verifies mandatory-field enforcement, successful record submission, reverse conditions, list-edit restrictions, and form-based updates through systematic testing.

Overall, the solution provides a lightweight and practical approach for improving **data quality, consistency, form usability, and validation** in ServiceNow Incident Management.

10. FUTURE SCOPE

The Implement Client Script & UI Policy (Incident) project can be further enhanced to provide more advanced validation, automation, and user-interface controls in ServiceNow.

1. **Advanced Validation** – Additional Client Scripts can be implemented to validate more Incident fields and enforce specific data-entry rules.
2. **Dynamic Field Control** – More UI Policies can be added to dynamically control field visibility, mandatory status, and read-only behavior based on different Incident conditions.
3. **Additional Automation** – Client-side automation can be extended to automatically populate or update related Incident information based on user selections.
4. **Enhanced Error Handling** – More user-friendly validation messages and guidance can be introduced to help users correct invalid or incomplete information.
5. **Integration with Server-Side Validation** – Client-side controls can be combined with server-side mechanisms to provide stronger and more consistent validation.
6. **Reporting and Monitoring** – The project can be extended with reports and dashboards to monitor Incident data quality, validation failures, and field usage.
7. **Role-Based Controls** – Different form behaviors can be implemented based on user roles and access requirements.
8. **Automation with Business Rules and Workflows** – The current Client Scripts and UI Policies can be integrated with Business Rules, Flow Designer, and workflows for more comprehensive Incident automation.

These enhancements can extend the current solution from basic Incident form validation into a broader Incident-management automation solution while maintaining data consistency and usability.

11. APPENDIX

11.1 Source Code

The project uses ServiceNow Client Scripts to implement dynamic Incident form behavior and validation.

A. onChange Client Script – Auto Set Urgency

```

function onChange(control, oldValue, newValue, isLoading) {

    if (isLoading || newValue == "") {

        return;

    }

    if (newValue == '1') {

        g_form.setValue('urgency', '1');

        g_form.addInfoMessage('Urgency set to High for High impact incident.');

```

B. onSubmit Client Script – Prevent Save if Assigned To is Missing

```

function onSubmit() {

    if (g_form.getValue('impact') == '1' &&

        g_form.getValue('assigned_to') == "") {

        g_form.showErrorMessage(

            'assigned_to',

```

'Assigned To is mandatory for High impact incidents.'

);

return false;

}

return true;

}

C. onCellEdit Client Script – Prevent State Change via List Edit

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
```

```
    alert('State cannot be updated using list editing. Please open the Incident.');
```

```
    callback(false);
```

```
}
```

The above scripts are based on the Client Script configurations specified in the project document.

11.2 Dataset Link

Dataset: Not Applicable (N/A)

This project does not use an external dataset. It works with **ServiceNow Incident records** and their configured fields, UI Policies, and Client Scripts.

11.3 GitHub Link

GitHub Repository: To be added

<https://github.com/Gokul27-star/Implement-Client-Script-UI-Policy-Incident-.git>

11.4 Project Demo Link

Project Demo: To be added

<https://drive.google.com/drive/folders/1SIJACHB8hzZHonB4ocRih1kjPIYxI6Yu?usp=sharing>